

Tracing Conditionals and Return

Lecture 13

CS 1001

Kirk Duran

Today

- ▶ Trace conditional control flow inside a function call.
- ▶ Explain why a function may contain several `return` statements but one call follows only one return path.
- ▶ Analyze chained and nested conditionals that return different values.
- ▶ Use early returns to terminate a call as soon as its result is known.
- ▶ Trace a function that calls a predicate function in its condition.
- ▶ Identify fall-through paths, implicit `None`, and unreachable statements.

Key idea

A conditional chooses a path; `return` terminates the current function call and sends one value back to its caller.

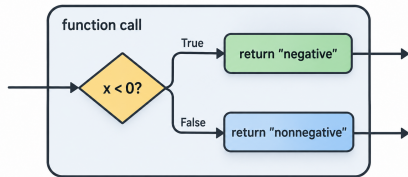
Part 1: Conditional Paths Inside a Function Call

Two Control-Flow Mechanisms Now Interact

- ▶ A conditional decides which suite executes next.
- ▶ A function call establishes a local execution environment.
- ▶ A return statement ends that call immediately.

Example

```
def sign(x):  
    if x < 0:  
        return "negative"  
    return "nonnegative"
```



return Is a Control-Flow Statement

Definition

Executing `return` expression evaluates the expression, terminates the current function call, and makes the resulting value the value of that call.

Example

```
def absolute_value(x):  
    if x < 0:  
        return -x  
    return x
```

Key idea

`return` does not merely leave the current `if` suite; it ends the entire active function call.

Multiple return Statements Are Alternative Exit Points

Source code

```
def classify(n):  
    if n < 0:  
        return "negative"  
    if n == 0:  
        return "zero"  
    return "positive"
```

- ▶ The definition contains three syntactic return statements.
- ▶ A particular call reaches at most one of them.
- ▶ Once one executes, the remaining body is not evaluated during that call.

Trace Example: Conditional Return

Program

```
def sign(x):  
    if x < 0:  
        return -1  
    return 1  
  
result = sign(-4)
```

Step	Execution	State / result
1	call sign(-4)	x -> -4
2	evaluate x < 0	True
3	execute return -1	call produces -1
4	resume caller	result -> -1

Statements After an Executed `return` Are Skipped

Program

```
def example(x):  
    if x > 0:  
        return x * 2  
        print("unreachable on this path")  
    return 0
```

- ▶ For `x > 0`, the first `return` ends the call.
- ▶ The following `print` cannot execute on that path.
- ▶ For `x <= 0`, execution skips the suite and reaches `return 0`.

Tracing rule

After an executed `return`, immediately remove the active call from the trace and resume the caller.

Part 2: Chained Conditionals with Return Values

A Chained Conditional Can Select a Return Value

Example

```
def letter_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    else:  
        return "below C"
```

Key idea

The chain chooses one suite; that suite's return determines the value of the call.

Trace a Chained Return Top to Bottom

Call

`letter_grade(84)`

1. Bind `score` -> 84.
2. Evaluate `score >= 90`: False.
3. Evaluate `score >= 80`: True.
4. Execute `return "B"`.
5. Terminate the call; the later tests are not evaluated.

Result

`letter_grade(84) → "B"`

Order Still Matters in a Chained Return

Correct ordering

```
if score >= 90:  
    return "A"  
elif score >= 80:  
    return "B"
```

- ▶ A score of 95 satisfies both predicates.
- ▶ An elif chain selects the first truthy branch.
- ▶ Later branches cannot repair a classification already returned.

Incorrect ordering

```
if score >= 80:  
    return "B"  
elif score >= 90:  
    return "A"
```

An Exhaustive Chain Guarantees an Explicit Return

Exhaustive

```
if n < 0:
    return "negative"
elif n == 0:
    return "zero"
else:
    return "positive"
```

- ▶ Every possible numeric input selects one branch.
- ▶ Every selected branch contains a return.
- ▶ Therefore every call reaches an explicit return value.

Independent ifs with Early Returns Behave Differently

Example

```
def classify(n):  
    if n < 0:  
        return "negative"  
    if n == 0:  
        return "zero"  
    return "positive"
```

- ▶ If the first condition is false, execution continues to the second if.
- ▶ If either early return executes, the later tests are never reached.
- ▶ The final return handles the remaining path.

Key idea

Early returns can make several independent tests behave like mutually exclusive exit paths.

Prediction: Which Return Executes?

Function

```
def shipping(weight):  
    if weight <= 0:  
        return "invalid"  
    elif weight <= 5:  
        return "standard"  
    elif weight <= 20:  
        return "heavy"  
    return "oversize"
```

In class

Predict the returned value for -2, 5, 12, and 25. For each call, identify exactly which conditions are evaluated.

Part 3: Nested Conditionals and Early Returns

A Return Inside a Nested Conditional Ends the Whole Call

Example

```
def admission(age, has_id):  
    if age >= 18:  
        if has_id:  
            return "admit"  
        return "need id"  
    return "underage"
```

- ▶ The outer condition is evaluated first.
- ▶ The inner condition is evaluated only when the outer condition is true.
- ▶ Any executed return terminates admission, not merely the nearest conditional.

Trace Nested Returns from the Outside In

Call

`admission(21, False)`

Step	Evaluation	Outcome
1	bind parameters	age -> 21, has_id -> False
2	age >= 18	True
3	has_id	False
4	return "need id"	call terminates

Returned value

"need id"

Nested Structure Is Appropriate for Dependent Questions

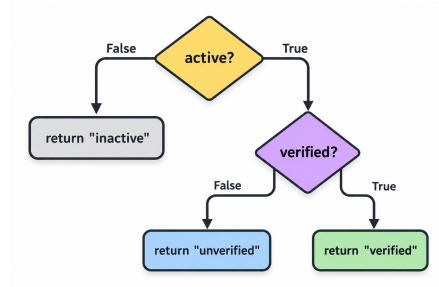
- ▶ Sometimes a second question is meaningful only after the first condition succeeds.
- ▶ Nesting represents this dependency directly.
- ▶ The inner condition is unreachable when the outer condition is false.

Logical structure

Is the account active?

↓ yes

Has the account completed verification?



Early Returns Can Reduce Unnecessary Nesting

Nested form

```
if amount >= 0:
    if amount <= limit:
        return amount
    else:
        return limit
else:
    return 0
```

Early-return form

```
if amount < 0:
    return 0
if amount > limit:
    return limit
return amount
```

Key idea

Equivalent behavior can sometimes be expressed with shallower control flow once invalid or exceptional cases return early.

A Guard Clause Rejects a Case Before the Main Path

Definition

A **guard clause** is an early conditional return that handles a case before the main computation continues.

Example

```
def ratio(total, count):  
    if count == 0:  
        return None  
    result = total / count  
    return result
```

- ▶ The guard handles the invalid denominator.
- ▶ The division is evaluated only on the remaining path.

Trace Example: Guard Clause

Call A

```
ratio(10, 0)
```

```
count == 0 → True
```

```
return None
```

Division is never evaluated.

Call B

```
ratio(10, 2)
```

```
count == 0 → False
```

```
result -> 5.0
```

```
return 5.0
```

Part 4: Predicate Calls as Conditions

A Predicate Call Can Determine the Branch

Definition

A **predicate function** returns a Boolean value representing a claim about its arguments.

Example

```
def is_even(n):  
    return n % 2 == 0  
  
def describe(n):  
    if is_even(n):  
        return "even"  
    return "odd"
```

Key idea

The predicate call must finish and produce a truth value before the surrounding `if` can select a branch.

Trace the Predicate Call Before the Conditional Branch

Call

`describe(7)`

1. Enter `describe`; bind `n` $\rightarrow$ 7.
2. Evaluate the condition `is_even(n)`.
3. Call `is_even(7)`; its local `n` $\rightarrow$ 7.
4. Evaluate `7 % 2 == 0` $\rightarrow$ `False`.
5. Return `False` to `describe`.
6. The `if` condition is false; execute `return "odd"`.

The Two Calls Have Separate Parameter Bindings

describe frame

n -> 7

waiting for condition value while is_even executes

is_even frame

n -> 7

evaluates predicate and returns False

Key idea

Identical parameter names in separate active calls represent distinct bindings in distinct call frames.

A Predicate Can Hide a More Detailed Test

Predicate

```
def is_valid_score(score):  
    return score >= 0 and score <= 100
```

Classification function

```
def result_label(score):  
    if not is_valid_score(score):  
        return "invalid"  
    if score >= 70:  
        return "passing"  
    return "not passing"
```

Trace Across the Predicate Boundary

Call: `result_label(105)`

1. `score` -> 105 in `result_label`.
2. Evaluate `is_valid_score(score)`.
3. Predicate returns `False`.
4. `not False` evaluates to `True`.
5. Execute `return "invalid"`.
6. No later condition in `result_label` is evaluated.

In class

Repeat the trace for `result_label(82)` and identify both returned values: first from the predicate, then from the classification function.

Predicate Results Participate in Ordinary Boolean Expressions

Example

```
if is_valid_score(score) and score >= 90:  
    return "high valid score"
```

- ▶ The predicate call is an expression whose value is Boolean.
- ▶ The larger condition follows the ordinary rules for Boolean expressions.
- ▶ To trace accurately, reduce function-call expressions to returned values before deciding the branch.

Expression reduction

```
is_valid_score(95) and 95 >= 90  
    → True and True  
    → True
```

Part 5: Fall-Through, Implicit Return, and Trace Errors



Not Every Conditional Function Returns on Every Path

Problematic definition

```
def category(n):  
    if n > 0:  
        return "positive"
```

- ▶ `category(5)` reaches the explicit return.
- ▶ `category(0)` skips the suite and reaches the end of the function body.
- ▶ Reaching the end without executing `return` produces `None`.

Tracing question

For every possible path through a value-producing function, ask: *where does this path return?*

Trace Fall-Through Explicitly

Call

```
answer = category(0)
```

Step	Execution	Result
1	enter category	n -> 0
2	evaluate n > 0	False
3	reach end of function	implicit None
4	resume caller	answer -> None

print **Still Does Not Replace** return

Prints a classification

```
def classify(n):  
    if n > 0:  
        print("positive")  
    else:  
        print("nonpositive")
```

Returns a classification

```
def classify(n):  
    if n > 0:  
        return "positive"  
    else:  
        return "nonpositive"
```

Key idea

Conditional output and conditional return values are different program behaviors.

Unreachable Code Can Be Path-Specific or Unconditional

Path-specific

```
if x > 0:  
    return x  
    print("never here")  
return 0
```

The print is unreachable only after the true branch is selected.

Unconditionally unreachable

```
return x  
print("never")
```

Once the first statement executes, no path reaches the second statement.

Common Tracing Error: Continuing After Return

Program

```
def fee(age):  
    if age < 5:  
        return 0  
    if age < 18:  
        return 8  
    return 12
```

In class

Trace `fee(3)`. Which conditions are actually evaluated? Then trace `fee(10)` and `fee(25)`.

Rule

Never continue tracing the body of a call after one of its return statements has executed.

Part 6: Integrated Tracing

Integrated Trace: Chaining Inside a Function

Program

```
def shipping_cost(weight):  
    if weight <= 0:  
        return None  
    elif weight <= 5:  
        return 6.0  
    elif weight <= 20:  
        return 12.0  
    return 20.0  
  
cost = shipping_cost(14)
```

In class

Produce the complete trace: parameter binding, every evaluated condition, selected branch, returned value, and final caller binding.

Integrated Trace: Predicate Call in the Condition

Program

```
def is_adult(age):  
    return age >= 18  
  
def ticket_price(age):  
    if is_adult(age):  
        return 15  
    return 10  
  
price = ticket_price(16)
```

- ▶ Trace both active call frames when `is_adult` executes.
- ▶ Record the predicate's returned value separately from `ticket_price`'s returned value.
- ▶ Finish by recording the caller binding for `price`.

Integrated Trace: Nested Conditionals with Multiple Returns

Program

```
def access_level(active, admin, verified):  
    if active:  
        if admin:  
            return "administrator"  
        elif verified:  
            return "member"  
        else:  
            return "limited"  
    return "inactive"
```

In class

Trace the conditions and return path for `access_level(True, False, True)` and `access_level(False, True, True)`. Do not evaluate conditions that execution never reaches.

A Reliable Manual Tracing Procedure

1. Enter the function and record parameter bindings.
2. Execute statements in order.
3. When a condition is reached, evaluate its expression completely.
4. Follow only the selected branch.
5. If a function call appears inside the condition, trace that call to its returned value first.
6. When `return` executes, evaluate its expression and terminate the current call immediately.
7. Resume the caller using the returned value.

Final Prediction

Program

```
def in_range(x):  
    return x >= 0 and x <= 10  
  
def transform(x):  
    if not in_range(x):  
        return None  
    if x < 5:  
        return x * 2  
    return x + 10
```

In class

Without running Python, determine `transform(-2)`, `transform(4)`, and `transform(8)`. For each call, list the exact sequence of function calls, conditions, and returns.

Takeaways

- ▶ A conditional selects an execution path; `return` terminates the active function call.
- ▶ Several `return` statements represent alternative exits, not several results from one call.
- ▶ Chained conditionals select the first applicable return path; nested conditionals represent dependent decisions.
- ▶ Early returns and guard clauses can simplify control flow when a result is already known.
- ▶ Predicate calls must be evaluated to Boolean values before their surrounding conditions can be resolved.
- ▶ A complete trace records parameter bindings, evaluated conditions, selected branches, return values, and the caller's resumed state.